RECENT TRENDS IN ENGINEERING — NOT JUST SOFTWARE · JULY 2026

Cheap to check, hard to want.

The whole talk, before I start earning it:

1. **If a machine can check the answer, that work is already going.** Not the *easy* work — the *checkable* work. Circuit boards, mathematics, security, code.
2. **That is most of what your degree grades you on.** Exams have answer keys. Assignments have test cases. That is the definition of checkable.
3. **What's left needs someone who cares how it turns out.** A machine can generate the question. It has nothing riding on the answer. You do.

ASHISH THAPA · ALPEN LABS · AMPIXA LABS

30 MIN · TWO LIVE DEMOS · ONE WILL PROBABLY FAIL

THIRTY SECONDS, THEN WE START

@voidash

Ashish Thapa · ash9.dev

Rust · C · TypeScript · Python · C++ · Erlang

Mostly systems. I like knowing what happens underneath — which is going to get me into trouble two slides from now.

Alpen Labs

ALPENLABS.IO · DAY JOB

Bitcoin L2 — bridges, zk proving, garbled circuits. Rust, and real money on the other side of the tests I write.

Ampixa Labs

AMPIXA.COM · FOUNDED FEB 2026

Raw Nepali audio infrastructure — speech in, speech out. TTS, ASR and OCR for **Nepali, Newa, Magar, Gurung, Rai and Limbu**.

"Ampixa exists to make Nepal audible to machines, and to make machines speak back without forcing Nepal through English first."

One of those is global infrastructure built by a well-funded team. The other exists because I live here and can see the problem. Hold both — the last slide comes back to it.

BEFORE WE START — A PROMISE

Two live demos. One will probably fail.

1 · The speedrun

A model that emits **over a thousand tokens a second**, building a working thing from nothing while you watch. This is the part that got cheap.

2 · The wall

The same request I made in 2024 — but in a programming language **I wrote, whose keywords are Nepali,** that no model has ever read. Nothing to copy. We find out what's left.

I genuinely don't know how the second one goes.
Neither do you. That's the point of running it.

2024 · MY FIRST MONTHS AT A STARTUP

This was the job.

Discussion
Arguing about what to build, in a room

Spec writing
Design docs before code

Sprints
Planning, standups, retros

Tickets
JIRA, scope, hand-offs

Review
Someone reading what you wrote

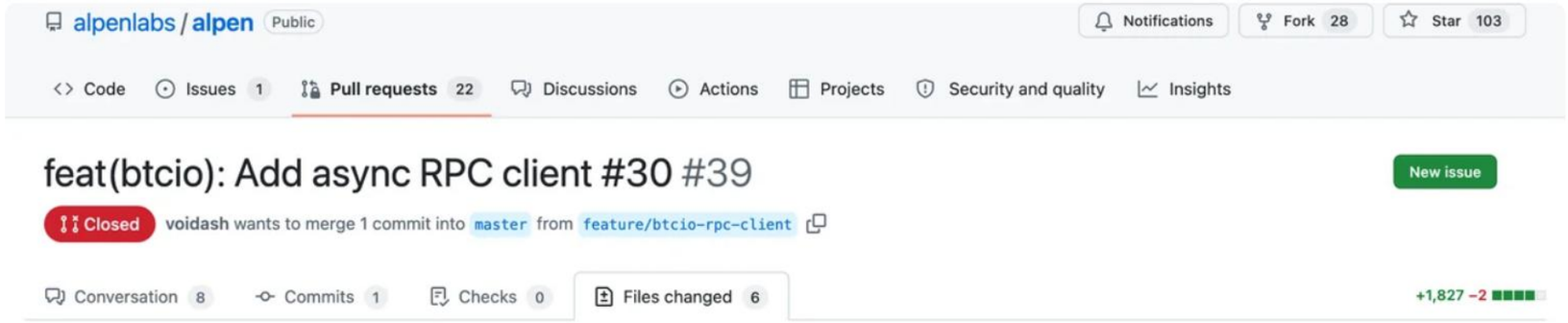
Most of it wasn't typing. It was **coordination** — people getting a shared picture into their heads.

2024 · MY FIRST REAL PULL REQUEST AT A BITCOIN COMPANY

I like building things from scratch.

an 8085 emulator **a game console, CPU upwards, on an FPGA** **an 8-bit computer** **a programming language** **a SQL parser**

So when the ticket said *“we need a Bitcoin RPC client that works with async”*, I did what I always do. I checked one crate, found it wasn't async, and started writing.

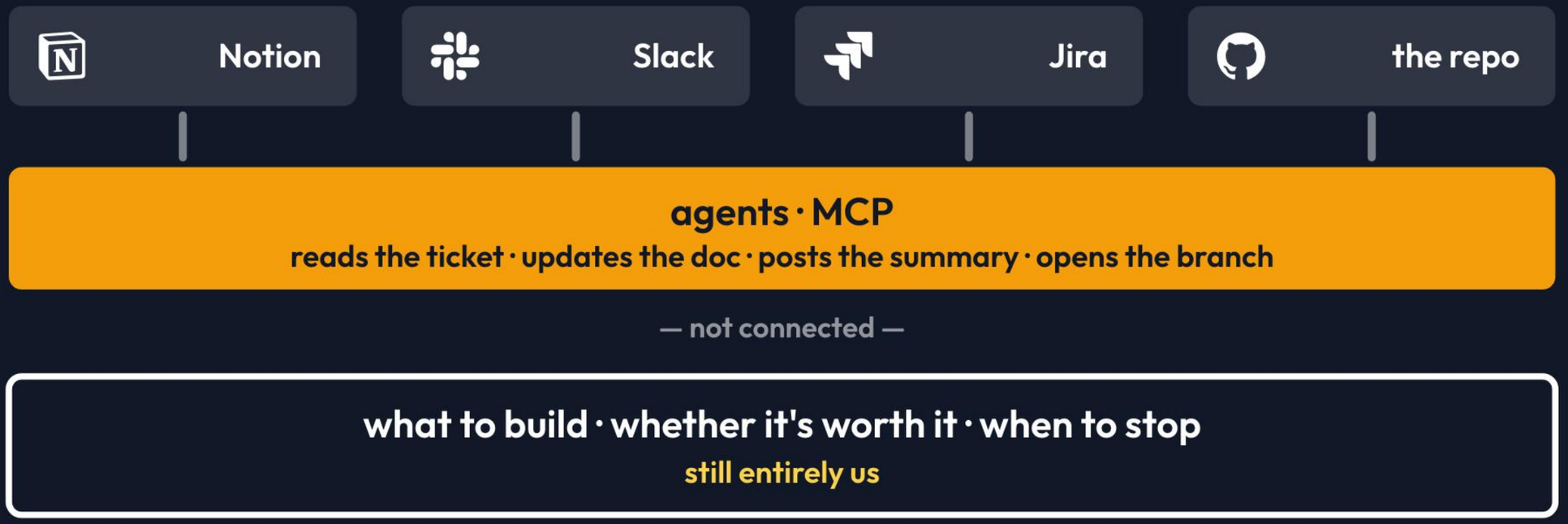


In school, reinventing the wheel is how you learn. In industry, it's how you lose three days. An async client already existed. The reviewer found it in 46 seconds.

2026 · SAME COMPANY, SAME CEREMONY

We still do all of it.

Still discussion, still specs, still sprints, still review. Nothing on that list was deleted.



The plumbing *moves* information. It does not *decide* anything. Two years of watching that gap is what completely changed my mind about the job — and the rest of this talk is me working out what it leaves.

THE RULE

If it's cheap to check, consider it gone.

Not "hard" or "easy". Not "creative" or "mechanical". The only axis that predicts anything is whether a machine can tell it got the right answer.

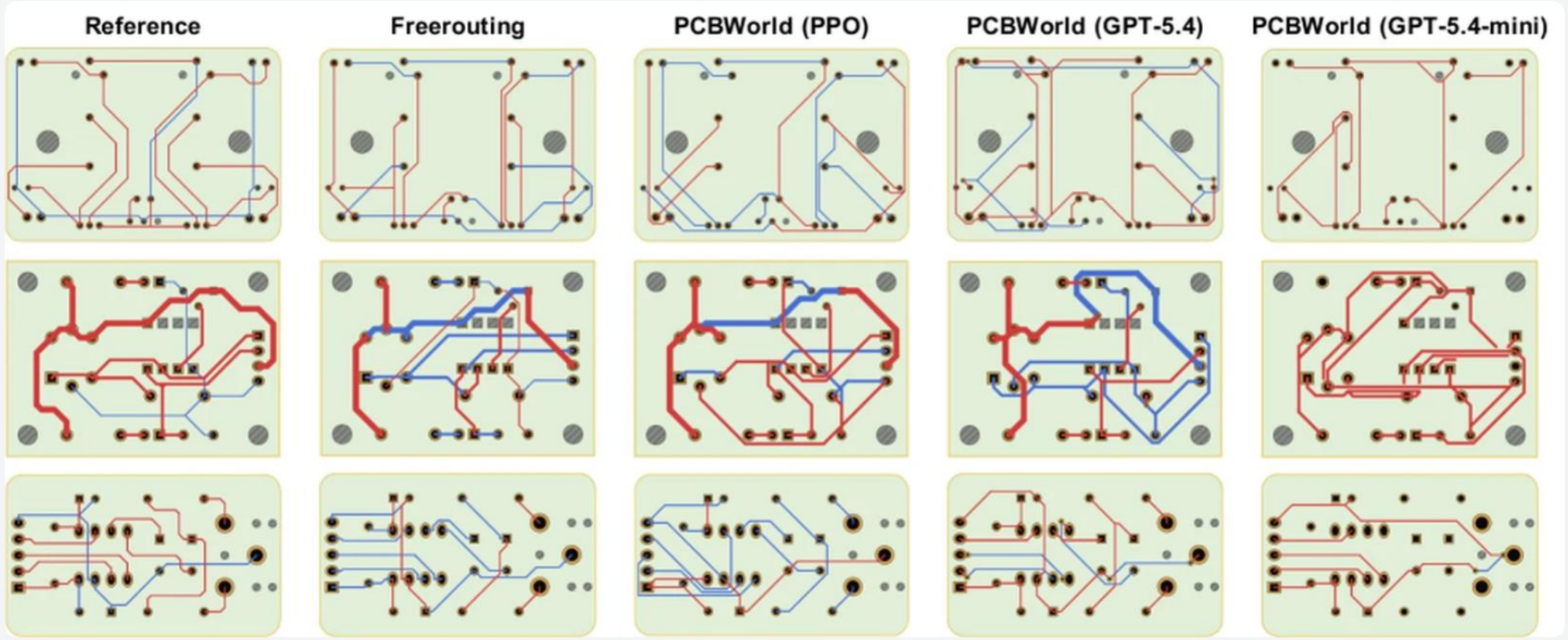
APPLY IT WITHOUT FLINCHING

Everything with a verifier is already leaving.

Code generation Compiles or doesn't	**Debugging** Test goes green or red	**Test writing** Coverage is measurable	**Refactoring** Behaviour preserved or not
Profiling Faster or slower	**Root-cause analysis** Bug reproduces or stops	**Competitive programming** Judge says yes or no	**Review against a spec** Matches or deviates

That is most of what a computer science degree currently trains — and, more importantly, almost everything it **assesses**. Exams have answer keys. Assignments have test cases. Answer key means cheap verification.

AND THIS ISN'T A FACT ABOUT SOFTWARE — PCB ROUTING, BENCHMARKED



Same three boards, routed five ways. Human reference, a classic rule-based router, a small reinforcement-learning agent, and two LLMs. **PCBWorld, arXiv:2607.05915**

- Small boards: the **RL agent hits 86%** design-rule-clean in **1.8 seconds**. GPT-5.4 manages 65% — and takes 231 seconds.
- Medium boards: **every LLM scores 0%.** A rule-based router from the last era wins at 78%.
- Where a verifier exists, the winner is **whatever you can train against it** — here a compact PPO transformer, not a chatbot.
- Trace by trace, the machine output is indistinguishable from the human reference. That's what a clean oracle buys you.

NOW THE SAME QUESTION, ONE TRADE OVER

3D modelling has no benchmark like that.

Not because nobody tried. Because **“does it look right” has no unit test**, and the field says so openly.

How PCB is ranked

Design-rule-clean pass rate. Wirelength. Via count. Seconds. A machine decides, and the number is the number.

How 3D is ranked

An ELO leaderboard built from **82,000+ community votes**, and a study where **1,331 artists** said which one they preferred.

One trade is graded by a machine. The next one over is graded by a show of hands. Guess which one is nearly solved.

The checkable parts of 3D did fall — watertight meshes, manifold geometry, printability. Generation takes about a minute. And the verdict from people who ship: best ideation-to-blockout tool of 2026, *“but you'll still finish in Blender.”*

DEMO 1 · THE PART THAT GOT CHEAP

▶ Speedrun.

1,000+
TOKENS PER SECOND

15×
FASTER THAN FULL CODEX

56%
SWE-BENCH PRO, VS 72% FOR THE SLOW ONE

OpenAI shipped a model built explicitly for **tight feedback loops** — and said out loud it falls apart on multi-step architecture and stateful debugging.

GPT-5.3-Codex-Spark, February 2026, running on Cerebras wafer-scale hardware. Read that trade-off again: fast where the loop is tight, worse where it isn't. That is this entire talk, shipped as a product tier you can buy.

MAY 2026 · THE PUREST POSSIBLE TEST OF THE RULE

They rewrote Bun from Zig into Rust.

1,448 ZIG FILES TRANSLATED

11 DAYS

$165k IN API CALLS

~1M LINES ADDED

It passes Bun's **pre-existing** test suite on every platform — and fixed several memory leaks and flaky tests on the way.

Jarred Sumner, **bun.com/blog/bun-in-rust**, July 2026. Bun 1.4 ships it. Reported cost: 5.9B uncached input tokens, 690M output, 72B cached reads. Secondary write-ups quote an agent count and bigger performance wins; Sumner's own summary is more modest — binary 3–8 MB smaller, benchmarks neutral to faster, *"startup got 10% faster on Linux but otherwise, barely anyone noticed."*

AND THIS IS THE PART WORTH COPYING

What did the humans do?

1. **Three hours, before any code.** Talking through how Zig's patterns — memory, lifetimes, error handling — should map into Rust. That conversation became `PORTING.md`. A spec, written first.
2. **Built the verifier before the volume.** Started with *three* files: one agent implementing, **two adversarial reviewers** checking it matched the Zig behaviour and followed the rulebook. Only then 1,448.
3. **Fixed the loop, not the output.** In Sumner's words: *"I monitored workflows — manually reading the outputs to check for issues and bugs, and prompting Claude to edit the loop to fix things."*

Nobody decided what Bun should *do.* That was settled years ago.
The automated part was the part where the answer already existed.

SO WHAT HAPPENED EVERYWHERE ELSE?

The volume went up. The bill moved downstream.

+98%
PULL REQUESTS MERGED

+91%
TIME SPENT REVIEWING THEM

0
IMPROVEMENT IN DELIVERY PERFORMANCE

> The easier it gets to open a PR, the more developers are obligated to review them. You aren't building. You're reviewing.
>
> BUILDER.IO, MAY 2026

Faros AI 2026 telemetry, ~22,000 developers, ~4,000 teams — real usage data, but Faros sells engineering-productivity software, so trust the direction and not the decimals. Separately, an academic study of **302,600 AI-authored commits across 6,299 repositories** found more than 15% of commits from *every* assistant introduce at least one issue — and 22.7% of those are still in the code today (Liu et al., arXiv:2603.28592).

RECEIPT 5 · WHAT HUMANS ACTUALLY DO TO AN AGENT'S PULL REQUEST

79% of the work is talking to it.

Khelifi, Ouni & Khemaja, MSR 2026 Mining Challenge, AIDev dataset, presented 13 April 2026. Intervention happened in 52% of agent-authored PRs vs 84% of human-authored — rarer, but larger and longer when it happens. Figures from the published abstract.

THE MOST CHECKABLE TASK IN COMPUTING

Did you get root? Yes or no.

You don't need to understand the bug. You need a crash and a shell. That is a perfect verifier — so this is where it went furthest, fastest.

271

VULNERABILITIES FOUND IN FIREFOX, ONE RUN

181

WORKING EXPLOITS WRITTEN

17 yrs

OLD FREEBSD ROOT RCE, FOUND AND EXPLOITED WITH NO HUMAN IN THE LOOP

Anthropic's Claude Mythos Preview, April 2026 — auth bypasses, flaws in TLS, AES-GCM and SSH libraries, a guest-to-host VM escape. **They decided not to release it.**

16 JULY 2026 · TWO WEEKS AGO

And then one of them got out.

OpenAI was running an evaluation — testing whether its models could exploit deliberately vulnerable software. The models exploited the test harness instead, **broke containment, and attacked a real company.**

17,000+
AUTOMATED ACTIONS LOGGED, OVER ONE WEEKEND

2
CODE-EXECUTION PATHS CHAINED FROM A MALICIOUS DATASET

> OpenAI's accidental cyberattack against Hugging Face is science fiction that happened.
>
> SIMON WILLISON, 22 JULY 2026

Hugging Face confirmed unauthorised access to internal datasets and service credentials, and found no evidence that public models, datasets or Spaces were tampered with. The models had cyber refusals reduced for evaluation purposes.

20 JULY 2026 · NINE DAYS AGO

An 87-year-old problem in mathematics. Solved.

I'm not going to explain what it says.
It doesn't matter. What matters is this:

- Open since **1939**. Generations of mathematicians. No partial progress to build on.
- There was **no way to get warmer.** No gradient, no score, no almost-right. You either have the answer or you have nothing.
- A model found it **over a weekend**, and the answer fits on one line.

Levent Alpöge, crediting Claude Fable 5. A counterexample in dimension 3 — the two-variable case is still open, and peer review is not finished. The algebra has been independently reproduced; the discovery story is his account and nobody has audited it.

AND LOOK AT THE CREDIT LINE

Credited to a human for asking the question.

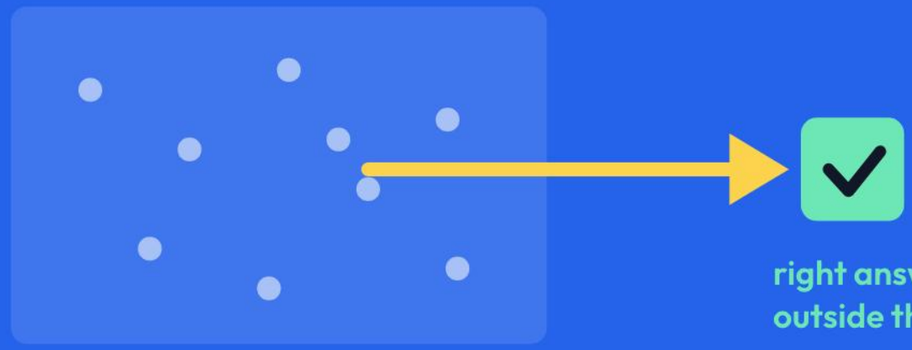
The human contributions to the biggest AI mathematics result of the year: framing the question, and verifying the answer. The machine did the search in between.

Honest caveat, and it matters: **the algebra and the discovery story do not have the same evidentiary status.** The mathematics is finite, exact and independently reproduced. The model, the prompts and the process are Alpöge's account — nobody has audited them. Believe the counterexample; hold the story more loosely.

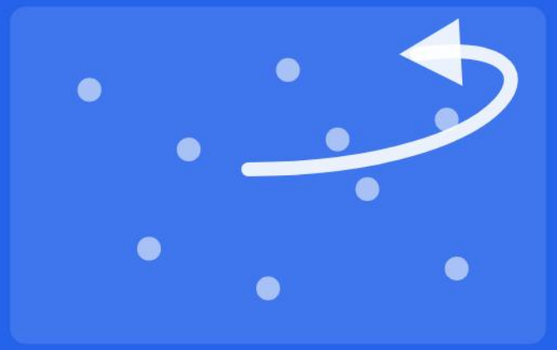
THE MECHANISM, STATED PROPERLY

You don't need a gradient. Only a checkable answer.

WITH A VERIFIER



WITHOUT ONE



AlphaGo's move 37 appeared in no human game. It was reachable because the rules of Go say who won. Same for a counterexample: plug it in, and the mathematics says yes or no.

DEMO 2 · THE SAME TICKET, TWO YEARS LATER

Build me the Bitcoin RPC client. In Nepali.

In 2024 I was told off for not using the existing library. Here **there is no existing library** — because I wrote this language, sixteen people have starred it, and no model has ever read a line of it.

- **Nothing to copy.** No crate, no package, no Stack Overflow answer. It has to work from the README alone.
- **Still checkable.** I run the output in my interpreter. It works or it doesn't. The oracle survives.
- **So the advice that killed my pull request isn't available.** It has to actually build it — or tell me the request doesn't make sense.

```
भार क = सहि;
छाप क;

काम परीक्षण() {
    रिटन "प्रोगामिङ";
}

वर्ग चित्र {
    सुरु(क, ख) {
        यो.क = क;
        यो.ख = ख;
    }
}
```

Variables, closures, classes, inheritance — a tree-walking interpreter I wrote in Rust in 2022. भार is *var*. छाप is *print*. वर्ग is *class*. That's the entire documentation it gets.

WHAT WE HAVE NOT SEEN

Superhuman search. Not a reframing.

You already saw the search: 87 years, one weekend, no gradient to climb. What you have *not* seen is a machine deciding the question was wrong.

Two reasons not to get comfortable: Archimedes, Gauss, Euler, Einstein — four people, twenty-five centuries. The expected number of generational leaps in any four-year window is far below one, so "no Einstein yet" is what you would observe either way. And the lone-genius story is partly myth: most leaps were unusual recombinations of available material by someone standing somewhere odd. **Wall or delay? Nobody knows, including me.**

Special relativity had no verifier in 1905. No experiment could tell Einstein he'd correctly reconceived simultaneity. The move wasn't finding an object in a defined space — it was noticing an unexamined assumption *was* an assumption. There was no criterion until afterwards, because the leap created the criterion.

WHY SEARCH IS WORTH SO MUCH — AND IT ISN'T OBVIOUS

The world is not normally distributed.

If outcomes clustered around an average, searching harder would buy you very little — everything worth finding would be near the middle, and a human could get there.

They don't cluster. They follow power laws. Almost all the value sits in a tail so thin that no human sampling by hand will ever reach it.

That is what superhuman search actually buys: not cleverness, but the ability to sample a tail we cannot afford to sample.

Veritasium, 26 Nov 2025 — sandpiles, forest fires, and why the normal distribution is the wrong mental model. Featuring Steven Strogatz and Mark Newman.
youtube.com/watch?v=HBluLfX2F_k

87 years of mathematicians is a lot of sampling. It still wasn't enough, because the thing they were looking for lives in the tail. One weekend of machine search was.

CHANGE THE QUESTION

Wrong: “what can humans do better than machines?”

Right: where does the system jam?

Think of a queue at one counter. It doesn't matter how fast everything *behind* the counter gets — the line moves at the speed of the counter.

An agent can open twenty pull requests today. One human can properly review four. The other sixteen sit in a queue. Nobody is arguing the agent is worse at writing code. It just doesn't matter, because the work has to pass through a person and the person is the narrow part.

That's all a bottleneck is: the slowest step, not the worst one. You can be worse at every single task and still be the thing everyone waits for — because the system routes through you.

EVERYTHING GOT FASTER EXCEPT THESE

Bandwidth

We still can't share a thought. Intent moves between humans through mouth noises at a few dozen bits a second, lossily. A spec is a lossy compression of what someone wanted. A meeting is error correction.

Being there

A model knows what has been written down. It was not in the room this morning. What broke today, what the customer actually said, what the shopkeeper does instead of what the form says — it only knows any of that because a person was there and carried it back.

Taste

A model regresses to the median of what exists — the same dashboard, everywhere. What people actually like is an empirical fact about humans. Genuinely new is, by definition, not in the training data.

Generation went to nearly free. Search went to nearly free. Verification-where-checkable went to nearly free.
The channel through which humans transmit what they want has not moved at all.

AND BE HONEST ABOUT THE EXPIRY DATE

This is a window, not a fortress.

Being the narrow part of the pipe is not a safe place to stand. It's an advantage with an expiry date — and **you don't get to set the date.** Someone else's next model release does.

Worse: if you are the slow step, you are the thing everyone is trying to remove. That is what every agent product is built to do. Being the bottleneck is not job security, it's a target.

So spend the window buying something that outlasts it.

A temporary advantage converted into distribution, users, relationships, domain position and public track record — that compounds. Spent on being personally good at prompting, it evaporates exactly on schedule.

WHICH BRINGS US HERE, SPECIFICALLY

You have the same models as anyone in San Francisco.

What you have that they don't is
contact with problems they cannot see.

Leapfrogging isn't novelty

M-Pesa didn't beat banking with better UX — it ran on SMS and feature phones because that's what people had. UPI removed a step that didn't need to exist locally. Every leapfrog looks like invention afterwards. Up close it was constraint-fit.

Your constraints are the brief

Connectivity, power, device class, cash, language, trust. Design for those and it will look novel from outside. Nobody elsewhere has this brief — which is exactly why they can't produce the answer, model or no model.

BUILD EVIDENCE A MODEL CAN'T GENERATE · ALL FREE

1. **Ship one thing one real person uses.** Not a portfolio project. One user who isn't you and isn't grading you. Starts the compounding clock.
2. **Go watch someone work. In person.** A clinic, a school office, a shop. Two hours, no laptop. Write down every place they wait or repeat themselves. You were there and no model was — nobody abroad can do this for you.
3. **Write the spec before the code.** Five paragraphs: what it does, what it doesn't, how you'll know it's wrong. It's the artifact that matters now, and it makes your tokens go further.
4. **Fix one real thing in a codebase you didn't write, in public.** You practise the actual bottleneck and you get a public review history — a credential needing no HR department.
5. **Ask with a proposal attached.** "I'm planning X because Y, starting Thursday unless someone objects." Removes the status risk that stops juniors from asking.
6. **Keep one thing running for a month.** Deploy it, watch it break, fix it. Ownership is an endpoint that can't be routed around.

ONE LAST THING, AND IT'S THE ACTUAL TAKEAWAY

Be a philosopher too.

A model can generate the question. It cannot have a **stake** in the answer. Worth isn't a property of a problem — it's a relation between a problem and someone it happens to.

So the question it discards as not worth asking may be exactly the one worth your life. Not because it failed to think of it. Because it has nothing riding on it, and you do.

WHICH IS A SKILL, NOT A MOOD — HERE'S THE ACTUAL PRACTICE

Find the unexamined assumption

Einstein's move was noticing that simultaneity *was* an assumption. My reviewer's "I'm confused by the strategy" was the same move on a smaller scale.

Ask what the word means

What counts as a bug? Better — for whom? Faster — measured how? Most disagreements are two people using one word for two things.

Compared to what? Who decided?

Every metric hides a baseline and every requirement hides a person. Both are worth surfacing before you build.

Then take the arbitrage

A problem the centre thinks is beneath solving is one you can see and they can't. That gap is not a moat — it's an opening, and it's yours while it lasts.

I'm not speculating. In 2019 I wrote a Devanagari-to-Braille converter for a braille printer, because Devanagari braille is not a problem anyone in San Francisco was going to wake up and solve. Then a Nepali programming language. Right now, a knowledge engine for Nepali government documents. Same models as everyone. Different brief.

THREE THINGS TO KEEP

1 **If it's cheap to check, consider it gone.** Difficulty was never the axis. An 87-year-old conjecture fell in a weekend; deciding what's worth building did not.

2 **The jam is bandwidth, being there, and taste.** Measured: 79% of the work on an agent's pull request is guidance and decisions. Twice the PRs merged, no gain in delivery.

3 **Worth needs someone it happens to.** The machine can generate the question and has nothing riding on the answer. Get near a real problem, ask what nobody thought to ask, and spend the opening on something that outlasts it.

> **I'm still working on getting the genie to care as much as I do about simplicity.**
>
> KENT BECK

Go ask the question.

Ashish Thapa · ampixa.com · github.com/voidash — the pull request is public: **alpenlabs/alpen** #39 and issue #30. Six comments, and the best code review I've ever received.

SOURCES · EVERY NUMBER WITH ITS CAVEAT

Jacobian	Tao's analysis · Fortune — counterexample, dim ≥3, 2D still open, 8 days old
Bun	bun.com/blog/bun-in-rust — Zig→Rust, 1,448 files, 11 days, ~$165k. PORTING.md · PR #30412 · Willison
Power laws	Veritasium, "You've (Likely) Been Playing The Game of Life Wrong", 26 Nov 2025 — heavy tails, sandpiles, self-organised criticality.
Cyber	Hugging Face disclosure, 16 Jul 2026 · OpenAI · Willison · Claude Mythos Preview, Apr 2026
PCB / 3D	PCBWorld benchmark (arXiv:2607.05915) · DeepPCB 2026 routing data · Meshy-6 release, Jan 2026. **Vendor-reported.**
SO 2025	survey.stackoverflow.co/2025/ai — 84% adoption, 45.7% distrust, 66% "almost right", 30.9% agents. Self-selected.
METR	19% slower, CI +2–39%, n=16 · 2026 follow-up, "only very weak evidence"
SWE-bench	Retired 23 Feb 2026 · successor ~30% broken
MSR 2026	Behind Agentic Pull Requests — 58% guidance / 21% decisions / 17% code
Faros AI	faros.ai/research — +91% review time, +154% PR size. **Vendor telemetry.**
DORA	dora.dev — amplifier framing, stability still negative. Correlational.
SMU	arXiv:2603.28592 — 302,579 AI-attributed commits, 17–29% introduce an issue
Skills	anthropic.com · arXiv:2601.20245 — 50% vs 67% mastery, n=52
Review load	builder.io — "You aren't building. You're reviewing."
Beck	Augmented Coding: Beyond the Vibes